WatchLog — Complete Beginner's Guide

Personal Movie & Book Watchlist — 9-Stage Learning Path

Author: Generated for ShritiSCeligoIO / WatchLog repo
Repository: https://github.com/ShritiSCeligoIO/watchlog
Last updated: July 2026
Current progress: **Stages 1–3 implemented** in code

How to use this guide

This document is written for someone **new to frontend development** (including QA engineers moving into dev).

- **Plain English first** — technical words are explained when they appear.
- **Stages 1–3** match your actual code and assignment briefs (**high confidence**).
- **Stages 4–9** are described from the **project end-state** and typical integrator-ui patterns (**verify against your official canvas** — exact brief text may differ).

Part 1 — The big picture

What is WatchLog?

WatchLog is a **personal watchlist app** for movies and books. You can:

- Search for titles using free public APIs
- Save items to your watchlist
- Track status: **Want → Watching/Reading → Done**
- Rate completed items (1–5 stars)
- Filter and sort your list

One codebase, nine stages

WatchLog is **not** nine separate apps. It is **one repo** where each stage adds a new layer:

```
Stage 1   Engine (TypeScript library, no UI)
Stage 2   First React UI (components, state, search)
Stage 3   Multiple pages (React Router, URLs)
Stage 4+  More production skills (state, persistence, styling, i1
```

Code from earlier stages **carries forward**. You refactor, not restart.

Final product goal (from project overview)

By Stage 9, WatchLog should feel **production-shaped**:

- Fully tested
- Internationalized (multiple languages)
- Deployable as a micro-frontend (MFE) like Celigo's integrator-ui

---

Part 2 — Beginner concepts glossary

Term	Simple meaning
TypeScript	JavaScript with types — catches mistakes before run
React	Library for building UI from reusable pieces (components)
Component	A function that returns what appears on screen (JSX)
Props	Data passed **into** a child component
State	Data that **changes over time** inside a component
Hook	Special React function starting with `use` (`useState`, `useEffect`)
Context	Shared data without passing props through every layer
Router	Maps browser URL → which page to show
API client	Code that talks to the internet (`fetch`)
Unit test	Automated check that one function behaves correctly
Fixture / seed data	Fake sample data for dev and tests

Part 3 — How to read any file (QA-friendly method)

Don't read top-to-bottom like a document. **Trace one user action.**

The 4-layer map

Every React screen usually has only four layers:

```
1. STATE     — what data exists? (useState, context)
2. HANDLERS  — what happens on click/type? (handleRemove, setQuer
3. RENDER    — what appears on screen? (return JSX)
4. CHILDREN  — who gets which props?
```

Example trace: "User clicks Remove"

```
1. User clicks Remove on card
2. WatchlistItemCard calls onRemove(item.id)
3. Parent's handleRemove filters item out of items state
4. React re-renders → card disappears
```

Practice this on every feature until it feels automatic.

Part 4 — Project structure (current repo)

```
WatchLog/
├── package.json          Scripts and dependencies
├── vite.config.ts        Dev server, build, tests
├── tsconfig*.json        TypeScript rules
├── .env.example          Environment variable template
│
└── src/
    ├── index.html        Empty page shell
    ├── index.tsx         Tiny entry → loads bootstrap
    ├── bootstrap.tsx     Starts React, router, error boundary
    ├── App.tsx           Route table
    │
    ├── pages/            Full screens (URLs)
    ├── components/       Reusable UI pieces
    ├── hooks/            Custom React logic
    ├── context/          Shared app-wide state
    ├── api/              HTTP clients (Stage 1)
    ├── utils/            Pure functions (Stage 1)
    ├── types/            Data shapes (Stage 1)
    ├── fixtures/         Sample/seed data
    ├── constants/        Fixed values (search length, filter nam
    ├── styles/           CSS
    └── config.ts         All environment variables
```

Entry chain (what runs when you open the app)

```
index.html
  → index.tsx (import bootstrap)
    → bootstrap.tsx (BrowserRouter + ErrorBoundary + mount React)
      → App.tsx (routes + providers)
        → AppLayout (header + nav + Outlet)
          → current Page (WatchlistPage, ItemDetailPage, etc.)
```

Part 5 — All nine stages

Overview table

Stage	Topic	Status in repo
1	TypeScript utility module	✅ Complete
2	React UI fundamentals	✅ Complete
3	React Router v6	✅ Complete
4	Global state (Redux)	🔜 Not started
5	Persistence (localStorage / store)	🔜 Not started
6	Advanced testing & quality	🔜 Partial (unit tests exist)
7	Styling / design system (e.g. Tailwind)	🔜 Basic CSS only
8	Internationalization (i18n)	🔜 Not started
9	Production / MFE deployment	🔜 Not started

> **Note:** Stages 4–9 titles are **[Likely]** based on project end-state and integrator-ui patterns. Confirm exact deliverables on your learning canvas.

Stage 1 — TypeScript Utility Module

In one sentence

Build the **engine**: typed data model, list utilities, and book search API — **no UI**.

Why Stage 1 exists

Real apps split **logic** from **UI**. Stage 1 is logic only. Stage 2+ import it instead of rewriting.

Assignment deliverables

Requirement	File
Movie + book data model	`src/types/watchlistItem.ts`
Filter by status	`src/utils/filterByStatus.ts`
Sort by rating	`src/utils/sortByRating.ts`
Group by genre	`src/utils/groupByGenre.ts`
Statistics summary	`src/utils/statsSummary.ts`
Async API + error handling	`src/api/openLibraryClient.ts`
Unit tests	`*.test.ts` beside source files
Bonus: movie search	`src/api/tmdbClient.ts`

Core concepts taught

1. Types and interfaces

WatchlistItem is a discriminated union — movie OR book:

```
type WatchlistItem = MovieWatchlistItem | BookWatchlistItem;
```

Use `item.type === 'movie'` to narrow which fields exist.

WatchStatus: `'want' | 'watching' | 'done'`

StarRating: `1 | 2 | 3 | 4 | 5` (only for done items)

2. Utility types

- `Partial<T>` — all fields optional (for updates)
- `Omit<T, 'id'>` — remove fields
- `Pick<T, 'title' | 'status'>` — keep only some fields

3. Pure functions (utils)

Input array → output array. No side effects. Same input = same output.

Example: `filterByStatus(items, 'done')` returns only done items.

4. Async API calls

```
searchBooks('dune')
  → build URL
  → await fetch(url)
  → parse JSON
  → return normalized results
  → throw on failure
```

Sync utils run instantly. **Async** API calls wait for the network.

5. Configuration

All environment variables live in **`config.ts`** — never scattered `process.env` reads.

6. Testing

Tests live **beside** source files (`filterByStatus.test.ts` next to `filterByStatus.ts`).

Run: `npm test`

Build library: `npm run build` → output in `dist/`

Stage 2 — React UI Fundamentals

In one sentence

Build a **single-page React app** that displays and edits watchlist data using **components, state, context, and a custom search hook — no Redux, no Router** (in original brief; Router added in Stage 3).

What Stage 2 teaches

Concept	What it means
JSX	HTML-like syntax inside JavaScript
Components	Reusable UI functions
Props	Pass data down
Events	Pass actions up (`onRemove`, `onAddBook`)
`useState`	Data that changes
`useContext`	Shared selection (later replaced by URLs in Stage 3)
`useEffect`	Run code when something changes
Custom hooks	Extract reusable logic (`useMediaSearch`)
`AbortController`	Cancel old search when user types again
ErrorBoundary	Catch render crashes

Key files (Stage 2 era → still relevant)

File	Role
`bootstrap.tsx`	Mount React
`components/SearchPanel.tsx`	Search UI
`components/WatchlistItemCard.tsx`	One list item
`hooks/useMediaSearch.ts`	Search logic

`components/ErrorBoundary.tsx`	Safety net
`utils/searchMappers.ts`	Turn API hit → watchlist item
`fixtures/seedWatchlist.ts`	Starting data

Props down, events up

```
Parent owns items state
  ↓ props: items, onAddBook
SearchPanel shows results
  ↑ event: onAddBook(result)
Parent adds to items
```

Child **never** owns the full list — parent does.

useState pattern

```
const [items, setItems] = useState(seedWatchlist);

// Remove — immutable update
setItems((prev) => prev.filter((item) => item.id !== id));

// Add — copy + append
setItems((prev) => [...prev, newItem]);
```

Never mutate with `push` — React may miss the change.

useMediaSearch flow

```
1. User types in search box → query state updates
2. useEffect sees query changed
3. If query too short → clear results
4. Else → AbortController cancels old request
5. await searchBooks(query)
6. setResults(data) or setError(message)
```

Search → Add → Card (E2E)

```
1. Click Add on search result
2. SearchPanel calls onAddBook(apiResult)
3. Parent calls bookSearchResultToWatchlistItem(result)
4. addWatchlistItem → setItems
5. New card appears
6. SearchPanel shows "Added" (Set lookup for speed)
```

ErrorBoundary

Catches **render** errors only — not click handlers or API failures.

- API errors → `useMediaSearch` sets `error` state
- Render bug → ErrorBoundary shows fallback message

Stage 3 — React Router v6 (Multi-page Navigation)

In one sentence

Split the app into **pages with URLs**, deep-link each item, guard the edit page, and store filters in the address bar.

Assignment deliverables (from brief)

#	Requirement	Implementation
1	3+ pages: list, detail, edit	`/watchlist`, `/items/:id`, `/items/:id/edit`
2	Deep-linkable item URLs	`/items/movie-1`
3	Auth guard → login	`ProtectedRoute` + `/login`
4	NavLink active state	`AppLayout` nav bar
5	Filters in URL (type + status)	`useSearchParams` via `useWatchlistFilters`

Routes (App.tsx)

```
/                    → redirect to /watchlist
/watchlist           → list + search + filters
/watchlist?status=want  → filtered list (survives refresh)
/items/:itemId       → detail page
/items/:itemId/edit  → edit form (protected)
/login               → mock sign-in
```

Layout + Outlet pattern

AppLayout = frame that never changes (header, nav).
Outlet = hole where the current page renders.

```
┌──────────────────────────┐
│ WatchLog  [Watchlist]    │  ← AppLayout
├──────────────────────────┤
│   <Outlet />             │  ← page swaps here
└──────────────────────────┘
```

useParams — read item id from URL

On `/items/movie-1`, detail page reads `itemId = 'movie-1'` and loads that item.

useSearchParams — sticky notes on URL

- Pathname = which room (`/watchlist`)
- Query string = filters (`?type=movie&status=want`)
- Refresh keeps query → filters survive

Default `'all'` when param missing — see `constants/watchlistFilters.ts`.

ProtectedRoute — bouncer

```
Not signed in + try /edit → redirect /login (remember where you c
Sign in → redirect back to /edit
```

Auth stored in `sessionStorage` (mock — closes when tab closes).

WatchlistDataContext

Items list moved to context so **all pages** share the same data.

Stage 4 — Global State (Redux) [Likely — verify canvas]

Why it comes after React basics

`useState` and Context work for small apps. Large apps (like integrator-ui) use **Redux** for predictable global state.

Expected topics

- Redux store, actions, reducers
- React-Redux `useSelector`, `useDispatch`
- Replace or complement `WatchlistDataContext`
- DevTools for debugging state changes

What you'd migrate

- Watchlist items CRUD
- Maybe auth state
- Maybe filter preferences

Stage 5 — Persistence [Likely — verify canvas]

Problem today

Refresh the page → watchlist resets to seed data (in memory only).

Expected solution

- Save items to **localStorage** or backend API
- Load on app startup
- "Persist across sessions" from project overview

Concepts

- Serialization (JSON.stringify / parse)
- Hydration on boot
- Optimistic UI updates

Stage 6 — Testing & Quality [Likely — verify canvas]

What you already have

- 92+ unit tests (Vitest)
- Component tests with Testing Library
- CI on GitHub Actions

Expected additions

- More integration tests (router + pages)
- Maybe E2E (Playwright/Cypress)
- Coverage targets
- Testing async flows and auth guards

Stage 7 — Styling / Design System [Likely — verify canvas]

Today

Basic CSS in `styles/stage2-layout.css`.

Expected

- Tailwind CSS or MUI (integrator-ui uses Fuse + MUI)
- Replace temporary CSS
- Responsive layouts, accessible components

Stage 8 — Internationalization (i18n) [Likely — verify canvas]

Goal from overview

App supports multiple languages.

Expected topics

- Translation files (en, es, etc.)
- `react-i18next` or similar
- No hardcoded UI strings
- Date/number formatting per locale

Stage 9 — Production / Micro-frontend [Likely — verify canvas]

Goal

"Production-shaped" app deployable like integrator-ui MFE.

Expected topics

- Production build optimization
- Environment-specific config
- Module federation or MFE shell integration
- Monitoring, error reporting
- Deployment pipeline

Part 6 — Environment & commands

Setup

```
npm install
cp .env.example .env
```

.env variables

Variable	Purpose
`TMDB_API_KEY`	Movie search (optional for books)

Commands

Command	What it does
`npm run dev`	Start app at localhost
`npm test`	Run all tests
`npm run build`	Compile Stage 1 library → `dist/`
`npm run build:app`	Production React bundle → `build/`

Part 7 — Common beginner questions

Why two TypeScript configs?

One builds the **library** (Node). One type-checks the **React app** (browser). Different rules.

Why bootstrap.tsx separate from index.tsx?

Matches integrator-ui pattern. Enables code splitting — tiny entry file loads first.

What's the difference between URL and state?

	URL	React state
Survives refresh	✅ (if in URL)	❌ (unless persisted)
Shareable link	✅	❌
Good for	pages, filters	form typing, toggles

Why immutable updates?

React compares references. Mutating arrays in place may not trigger re-render.

Part 8 — Study checklist by stage

Stage 1 — Can you explain?

- [ ] What is a discriminated union?
- [ ] What does `filterByStatus` return?
- [ ] Sync vs async — which is `searchBooks`?
- [ ] Why co-locate tests?

Stage 2 — Can you explain?

- [ ] Props down / events up — one example
- [ ] Why `useEffect` in search?
- [ ] What does AbortController do?
- [ ] ErrorBoundary vs search error message

Stage 3 — Can you explain?

- [ ] What does `<Outlet />` do?
- [ ] What is `:itemId` in a route?
- [ ] How do URL filters survive refresh?
- [ ] Why is edit page protected?

Part 9 — Quick reference — current URLs

URL	Page
`/watchlist`	Main list
`/watchlist?type=movie&status=done`	Filtered list
`/items/movie-1`	Arrival detail
`/items/movie-1/edit`	Edit (sign in required)
`/login`	Sign in

End of guide.
For the latest code, always trust the repo over this document. Update canvas briefs may change Stages 4–9 details.